# Лабораторная работа 6 Контейнеризация и управление средой

Методические материалы курса · учебный год 2026–2027

Версия: 2026–2027 учебный год Формат: команда 2–5 человек с индивидуальной сдачей Предшествующая работа: ЛР5 «Тестирование данных, модели и pipeline»

## 1 Цель

Упаковать проверенную ML-систему в воспроизводимый OCI-образ, запустить её через Docker Compose с ограниченными правами и подтвердить готовность и предсказание реальными HTTP-запросами. Научиться различать build-time и runtime, image и container, health и readiness, воспроизводимый tag и плавающий latest.

## 2 Результаты обучения

После выполнения каждый студент умеет:

- объяснить build context, layer cache и назначение .dockerignore;
- читать стадии Dockerfile и локализовать ошибку сборки;
- выполнить тесты до создания runtime image;
- запускать процесс от non-root пользователя;
- передавать runtime-конфигурацию без включения секретов в образ;
- объяснить HEALTHCHECK, /healthz и /readyz;
- получить image ID и связать его с commit;
- выполнить smoke test контейнера и диагностировать один отказ;
- доказать личный вклад через issue, PR, review и evidence card.

## 3 Командный результат

Команда предоставляет tag lr6-v1 и:

1. многостадийный Dockerfile с quality/build gate;
2. .dockerignore с минимальным build context;
3. compose.yaml без поля version;
4. runtime от non-root пользователя;
5. healthcheck по /readyz;
6. read-only filesystem, tmpfs, cap_drop и no-new-privileges в Compose;
7. образ mlops-flight-delay:lr6-<short-sha>;
8. reports/container/lr6-inspect.txt с image ID, размером, пользователем и healthcheck;
9. reports/container/lr6-smoke.json с ответом /predict;
10. таблицу «риск — ограничение — проверка»;
11. issue, pull requests, review и индивидуальные evidence cards.

latest не принимается как идентификатор результата. Секреты, токены и персональные данные не помещаются в Dockerfile, build arguments, labels, image layers или отчёты.

## 4 Архитектура сборки

| Стадия | Содержимое | Назначение |
|---|---|---|
| quality | исходный код, данные, tests, dev-зависимости | Ruff, pytest, контракт данных, обучение модели |
| runtime | runtime-зависимости, приложение, модель | минимальная среда исполнения API |

Runtime image должен зависеть от успешной quality-стадии. Если тесты падают, принимаемый образ не создаётся. Учебная модель строится внутри build для воспроизводимости; в производственной системе модель обычно поступает как проверенный внешний артефакт через release pipeline.

## 5 Роли

| Роль | Зона ответственности | Индивидуальное доказательство |
|---|---|---|
| Image engineer | стадии, cache и зависимости | diff Dockerfile и объяснение layers |
| Runtime security engineer | USER, filesystem и capabilities | Compose-ограничение и проверка inspect |
| Model artifact engineer | обучение и перенос модели | проверка bundle и readiness |
| Smoke-test engineer | health, predict и отчёт | скрипт или диагностический лог |
| Quality reviewer | .dockerignore, review и риски | содержательное review и defect injection |

Для 2–4 человек роли совмещаются. Роль должна отличаться от ЛР5; один студент не остаётся единственным владельцем модели или инфраструктуры более двух последовательных работ.

## 6 Предварительные условия

- Docker Engine или Docker Desktop с Compose v2;
- успешный python -m pytest -q из ЛР5;
- закрытые данные и секреты удалены из build context;
- свободен порт 8080 либо задан APP_PORT;
- команда работает в отдельной ветке от tag ЛР5.

```
git switch -c lab6/containerization
docker version
docker compose version
python -m pytest -q
```

Если Docker недоступен на личной машине, преподаватель может предоставить общий учебный runner. Скриншот чужой сборки не заменяет индивидуальную диагностическую операцию.

## 7 Build context и .dockerignore

Build context — набор файлов, доступных инструкции COPY. Чем он больше, тем медленнее передача и выше риск включить лишние данные. Минимальный .dockerignore исключает:

```
.git
.github
.venv
__pycache__
.pytest_cache
.ruff_cache
docs
k8s
models
reports
mlruns
mlartifacts
*.pyc
.env
```

Для учебной quality-стадии src, tests, data, pyproject.toml и README.md остаются в context. Проверьте список до build; .dockerignore не является механизмом удаления уже попавшего секрета из истории образа.

## 8 Многостадийный Dockerfile

Обязательные решения:

- явная версия Python 3.12 slim;
- отдельная quality-стадия;
- установка зависимостей до часто меняющегося кода, где это возможно;
- Ruff, pytest и validation перед runtime;
- обучение models/model.joblib с фиксированными параметрами;
- runtime без tests, data и инструментов разработки;
- non-root USER с явным UID/GID;
- exec-форма CMD;
- HEALTHCHECK без внешней утилиты curl.

Сборка:

```
SHORT_SHA=$(git rev-parse --short HEAD)
docker build --pull --tag mlops-flight-delay:lr6-${SHORT_SHA} .
```

В PowerShell:

```
$shortSha = git rev-parse --short HEAD
docker build --pull --tag "mlops-flight-delay:lr6-$shortSha" .
```

Повторите build без --no-cache и сравните строки CACHED. Затем измените только README и объясните, какие layers перестроились.

## 9 Почему non-root обязателен

Если процесс не требует административных прав, он не должен работать как root. Проверьте конфигурацию image:

```
docker image inspect mlops-flight-delay:lr6-<short-sha> --format '{{.Config.User}}'
```

Ожидается непустой non-root UID/GID. Это снижает последствия компрометации, но не заменяет обновление base image, ограничение capabilities и безопасную конфигурацию хоста.

## 10 Health и readiness

/healthz подтверждает, что процесс отвечает. /readyz подтверждает, что модель загружена и сервис способен принимать предсказания. Docker healthcheck ЛР6 использует /readyz, иначе контейнер без модели мог бы считаться готовым.

```
docker inspect --format '{{json .State.Health}}' <container-id>
```

Статус сначала может быть starting, затем должен стать healthy. Не заменяйте ожидание фиксированным длинным sleep; опрашивайте состояние до timeout.

## 11 Запуск через Docker Compose

```
docker compose up --build --detach
docker compose ps
docker compose logs --no-color app
```

В compose.yaml должны быть:

- build и локальный неизменяемый для работы tag;
- настраиваемый host port ${APP_PORT:-8080};
- healthcheck;
- read_only: true;
- tmpfs: /tmp;
- cap_drop: ALL;

- security_opt: no-new-privileges:true;
- runtime environment без секретных значений.

Не добавляйте верхнеуровневое поле version: современная Compose Specification в нём не нуждается.

## 12 Smoke test

После статуса healthy выполните:

```
python scripts/container_smoke.py --base-url http://127.0.0.1:8080 --output reports/container/
lr6-smoke.json
```

Скрипт обязан:

1. дождаться /readyz с ограниченным timeout;
2. отправить типизированный запрос /predict;
3. проверить HTTP status;
4. проверить диапазон delay_probability;
5. проверить непустые model_version и request_id;
6. сохранить компактный JSON без секретов.

Ручной curl полезен для диагностики, но принимаемый результат должен воспроизводиться одной командой.

## 13 Отчёт inspect

```
docker image inspect mlops-flight-delay:lr6-<short-sha> \
  --format 'id={{.Id}} size={{.Size}} user={{.Config.User}} health={{json .Config.
Healthcheck}}'
```

Сохраните команду и результат в reports/container/lr6-inspect.txt. Image ID идентифицирует локальный content-addressed объект; для итогового release будет использован registry digest.

## 14 Проверка отсутствия лишних файлов

```
docker run --rm --entrypoint python mlops-flight-delay:lr6-<short-sha> -c \
  "from pathlib import Path; print([p.name for p in Path('/app').iterdir()])"
```

В runtime не должно быть .git, tests, исходных данных, .env, MLflow-хранилища и отчётов. Наличие файла внутри удалённого поздней инструкцией layer не означает, что он исчез из истории; не копируйте лишнее изначально.

## 15 Негативные сценарии

Команда воспроизводит минимум два отказа:

| Дефект | Ожидаемый сигнал | Диагностика |
|---|---|---|
| модель не перенесена в runtime | /readyz возвращает 503, health unhealthy | проверить MODEL_PATH и COPY stage |
| процесс запущен на другом порту | health timeout | сопоставить CMD, EXPOSE и health URL |
| запрещена запись вне /tmp | операция завершается permission denied | проверить ожидаемый путь записи |
| порт хоста занят | Compose не запускает binding | изменить APP_PORT, не порт контейнера |
| .env попал в context | нарушение security review | исправить .dockerignore, сменить секрет |

Defect injection выполняется в отдельной ветке или незакоммиченной копии и не входит в tag lr6-v1.

## 16 Таблица рисков

| Риск | Ограничение | Проверка |
|---|---|---|
| непроверенный код попал в image | quality stage | намеренно падающий тест блокирует build |
| процесс получил лишние права | non-root, cap_drop, no-new-privileges | inspect и запуск команды id |
| образ содержит лишние данные | .dockerignore, выборочный COPY | просмотр /app и image history |
| сервис жив, но модель не готова | healthcheck по /readyz | Compose health status |
| результат нельзя связать с кодом | tag с short SHA и image ID | manifest inspect |
| runtime меняет файловую систему | read-only и tmpfs | контролируемая запись |

## 17 Индивидуальная сдача участника команды

Командный результат: tag lr6-v1, image mlops-flight-delay:lr6-<short-sha>, успешные inspect и smoke reports.

| Поле | Что указать |
|---|---|
| Студент | ФИО, группа, команда, GitHub-логин |
| Лабораторная | ЛР6, commit/tag и image ID |
| Роль | роль в ЛР6 и отличие от ЛР5 |
| Задача | issue с наблюдаемым критерием приёмки |
| Личный артефакт | Dockerfile/Compose diff, smoke test, review или отчёт |
| Вклад | созданное, исправленное или проверенное решение |
| Инженерное решение | выбранный вариант и отвергнутая альтернатива |
| Риск | отказ, который выявляет healthcheck, inspect или smoke test |
| Диагностика | команда, первый полезный фрагмент лога и причина |
| Использование ИИ | инструмент и самостоятельно проверенные утверждения либо отсутствие использования |

Минимальный допуск: командная версия зафиксирована, образ связан с commit, личная issue закрыта через PR, review выполнен другим участником, контейнер healthy и индивидуальная проверка воспроизводится.

## 18 Защита

За 3–4 минуты участник:

1. показывает image tag, ID и связанный commit;
2. объясняет одну стадию или runtime-ограничение;
3. запускает собственную проверку;
4. воспроизводит назначенный отказ;
5. находит причину в build log, container log или inspect;
6. исправляет конфигурацию либо формулирует точное исправление;
7. объясняет, почему image нельзя идентифицировать только словом latest.

Преподаватель может изменить APP_PORT, удалить модель из build stage, назначить проверку пользователя или попросить показать файл, исключённый из context.

## 19 Повторная сдача и стоп-факторы

Повторная сдача выполняется через новую или переоткрытую issue, отдельный PR, новый image tag и обновлённую evidence card. Неуспешный отчёт сохраняется.

Стоп-факторы:

- контейнер работает от root без обоснованного исключения;
- используются latest или несвязанный с commit tag;
- тесты не являются gate сборки;
- /readyz успешен без загруженной модели;
- в image или отчёте обнаружен секрет;
- студент демонстрирует чужой образ или не может объяснить layer;
- дефект остался в принимаемом tag;
- smoke result отредактирован вручную.

## 20 Критерии оценивания

| Критерий | Баллы | Полное выполнение |
|---|---|---|
| Build и воспроизводимость | 2 | стадии, test gate, фиксированный Python и связанный tag |
| Минимальный runtime | 2 | выборочный COPY, .dockerignore, нет tests/data/dev tools |
| Безопасность | 2 | non-root, read-only, tmpfs, capabilities и no-new-privileges |
| Health и smoke | 1 | ready healthcheck и автоматический predict report |
| Командный процесс | 1 | issue, небольшие PR и содержательное review |
| Индивидуальная защита | 2 | личный артефакт, отказ, диагностика и объяснение риска |

Максимум: 10 баллов. Рекомендуемый порог зачёта: 6 баллов при отсутствии стоп-факторов.

## 21 Чек-лист

- [ ] Команда состоит из 2–5 человек, роли ротированы.
- [ ] Build context не содержит секретов и лишних каталогов.
- [ ] Quality stage запускает Ruff, pytest и validation.
- [ ] Runtime image содержит обученную модель и приложение.
- [ ] Процесс работает от non-root UID/GID.
- [ ] Docker healthcheck проверяет /readyz.
- [ ] Compose использует read-only, tmpfs и security options.
- [ ] Образ помечен lr6-<short-sha>, а не latest.
- [ ] Inspect и smoke reports созданы командами.
- [ ] Каждый участник выполнил личную проверку и defect injection.
- [ ] PR прошёл review, создан tag lr6-v1.

## 22 Связь с ЛР7

В ЛР7 API-контракт будет расширен и проверен через OpenAPI, типизированные положительные и отрицательные запросы. Контейнерная граница ЛР6 остаётся неизменной: /healthz, /readyz, /predict, порт 8080 и модельный bundle являются совместным контрактом.

## 23 Официальные источники

- Dockerfile reference: https://docs.docker.com/reference/dockerfile/
- Docker build best practices: https://docs.docker.com/build/building/best-practices/
- Docker build context and .dockerignore: https://docs.docker.com/build/concepts/context/
- Docker Compose services: https://docs.docker.com/reference/compose-file/services/
- Docker Compose startup and health: https://docs.docker.com/compose/how-tos/startup-order/